

MISSÃO 03: O Escudo Protetor (Encapsulamento e .gitignore)

Objetivo: Aprender a proteger os dados do seu sistema contra "hackers" ou erros lógicos, dominar os atalhos do VS Code e impedir que a sua IDE mande "lixo" para o GitHub.

No mercado, um programador nunca deixa as portas do seu sistema abertas, e também nunca digita código repetitivo na mão. Vamos ativar o modo profissional da sua IDE!

PASSO 1: A Falha de Segurança (O Bug)

Vamos usar o projeto da Missão anterior para ver um problema grave acontecer.

1. Abra o VS Code. Vá em File > Open Folder... e abra a pasta do projeto `meu-primeiro-sistema` (da Missão 02).
2. Abra o arquivo `Carro.java` e adicione um novo atributo público:
`public int velocidade;`
3. Vá para o arquivo `Sistema.java` (na sua Main) e adicione esta linha logo abaixo de onde você definiu a cor do carro:
`meuCarro.velocidade = -500;`
4. Pense: Existe um carro que ande a "menos 500 km/h"? Não! Como deixamos a variável `public` (pública), qualquer um pode inserir dados absurdos no nosso sistema e quebrar a nossa lógica.

PASSO 2: Trancando as Portas (Private)

Vamos aplicar o Encapsulamento (um dos pilares da Orientação a Objetos).

1. Volte ao arquivo `Carro.java`.
2. Troque a palavra `public` de todos os atributos para `private`:

```
private String modelo;  
private String cor;  
private int velocidade;
```

3. Salve o arquivo (`Ctrl + S`).
4. Volte para o arquivo `Sistema.java`.

(🔴 Viu o que aconteceu? A sua tela deve estar cheia de erros vermelhos! Como os atributos agora são privados, a classe Sistema não tem mais permissão para encostar neles diretamente).

PASSO 3: A Mágica do VS Code (Getters e Setters)

Se os atributos estão trancados, como vamos colocar valores neles? Usando métodos públicos (as "catracas" de acesso). Você não vai digitar isso na mão!

1. Volte ao `Carro.java`.
2. Clique com o Botão Direito em qualquer lugar vazio do código e escolha Source Action... (Ação de Origem).
3. Selecione Generate Getters and Setters (Gerar Getters e Setters).
4. Marque a caixinha para selecionar todos os atributos (modelo, cor, velocidade) e dê OK.
(BOOM! O VS Code acabou de escrever umas 20 linhas de código perfeito para você em 1 segundo).

PASSO 4: Consertando a Main e Validando a Regra

Agora vamos usar as portas corretas e impedir o carro de ter velocidade negativa.

1. No `Carro.java`, procure o método `setVelocidade(int velocidade)`. Mude ele para adicionar uma regra de segurança:

```
public void setVelocidade(int velocidade) {  
    if (velocidade < 0) {  
        System.out.println("Erro: Velocidade não pode ser  
negativa!");  
    } else {  
        this.velocidade = velocidade;  
    }  
}
```

2. No `Sistema.java`, apague os erros vermelhos e troque por:

```
meuCarro.setModelo("Fusca");  
meuCarro.setCor("Azul");  
meuCarro.setVelocidade(-500); // Tentando hackear novamente
```

3. Rode o programa. (A mensagem de Erro que você criou deve aparecer e o sistema está salvo!).

PASSO 5: O Perigo Invisível da IDE

1. No terminal do VS Code, digite: `git status`
2. Além dos seus arquivos `.java`, você deve ver arquivos vermelhos terminados em `.class` (arquivos compilados) ou uma pasta chamada `.vscode`.
3. A Regra: Nós NUNCA mandamos esses arquivos para o GitHub. O GitHub só quer o código legível (`.java`).



PASSO 6: O Segurança da Balada (.gitignore)

1. Na RAIZ do seu projeto (fora da pasta src), crie um arquivo chamado `.gitignore` (Tem que ter o ponto no início!).
2. Digite exatamente isso dentro dele e salve:

```
.class  
.vscode/  
bin/
```

3. Digite `git status` novamente.
(Mágica! Os arquivos `.class` sumiram da visão do Git. O `.gitignore` bloqueou a entrada deles no palco!)



PASSO 7: Salvando o Progresso

Agora que o sistema está blindado e o lixo foi bloqueado, suba para a nuvem:

1. `git add .`
2. `git commit -m "feat: aplica encapsulamento e gitignore"`
3. `git push`

Missão 03 Concluída! Você aprendeu a validar regras de negócio trancando seus atributos e garantiu que seu repositório no GitHub fique digno de um profissional Sênior.